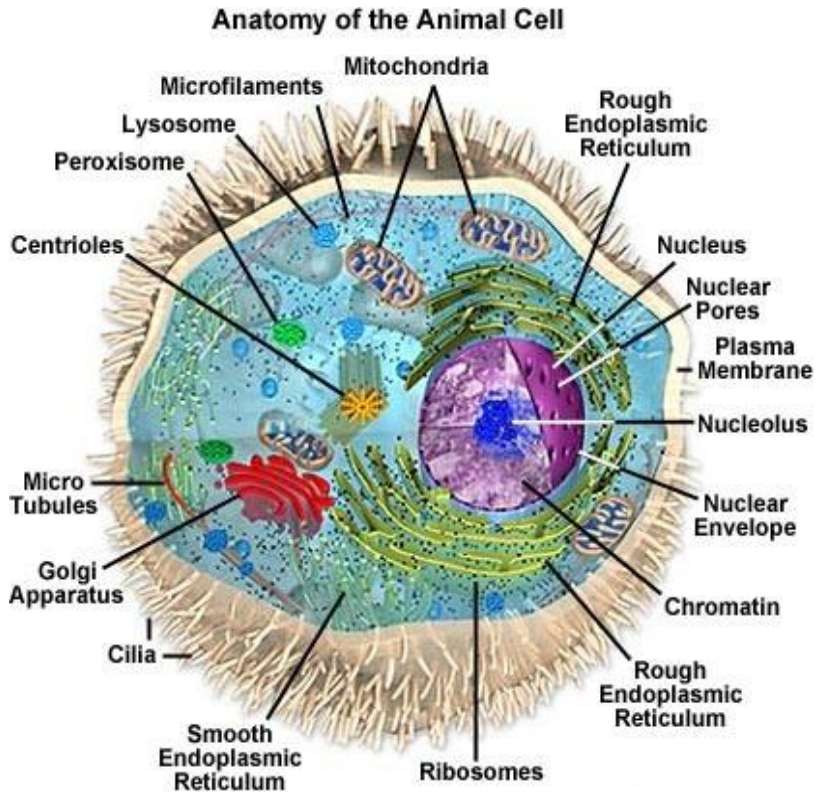
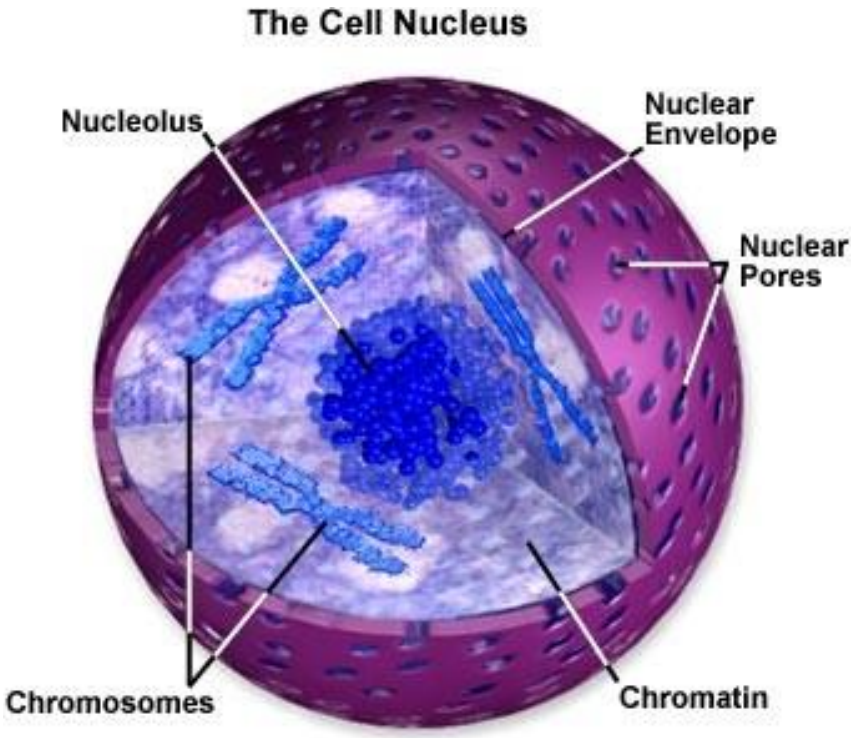
Genetic Algorithms

Introduction

- After scientists became disillusioned with classical and neo-classical attempts at modeling intelligence, they looked in other directions.
- Two prominent fields arose, connectionism (neural networking, parallel processing) and evolutionary computing.
- It is the latter that this lecture deals with - genetic algorithms.


Anatomy of the Animal Cell
Mitochondria
Microfilaments
Lysosome
Peroxisome
Rough Endoplasmic Reticulum
Centrioles
Nucleus
Nuclear Pores
Plasma Membrane
Nucleolus
Micro Tubules
Nuclear Envelope
Golgi Apparatus
Chromatin
Cilia
Rough Endoplasmic Reticulum
Smooth Endoplasmic Reticulum
Ribosomes


The Cell Nucleus
Nucleolus
Nuclear Envelope
Nuclear Pores
Chromosomes
Chromatin

Can evolution be intelligent?

- Intelligence can be defined as the capability of a system to adapt its behaviour to ever-changing environment.

- **We are products of evolution**, and thus by modeling the process of evolution, we might expect to **create intelligent behavior**.

- **Evolutionary computation simulates evolution on a computer**.

- The result of such a **simulation** is a series of optimization algorithms, usually based on a simple set of rules. Optimization iteratively **improves the quality of solutions until an optimal, or at least feasible**, solution is found.

Simulation of Natural Evolution

- On 1 July 1858, **Charles Darwin** presented his theory of evolution before the Linnean Society of London. This day marks the beginning of a revolution in biology.

- Darwin's classical **theory of evolution**, together with Weismann's **theory of natural selection** and Mendel's concept of **genetics**, now represent the **neo-Darwinian** paradigm.

- **Neo-Darwinism** is based on processes of **reproduction, mutation, competition and selection**.
  - The power to **reproduce** appears to be an essential property of life.
  - The power to **mutate** is also guaranteed in any living organism that **reproduces itself** in a continuously changing environment.
  - Processes of **competition and selection** normally take place in the natural world, where expanding populations of different species are limited by a finite space.

- Evolution can be seen as a process leading to the maintenance of a population's **ability to survive and reproduce in a specific environment**. This ability is called **evolutionary fitness**.

- Evolutionary fitness can also be viewed as a measure of the organism's ability to anticipate changes in its environment.

- The fitness, or the quantitative measure of the ability to predict environmental changes and respond adequately, can be considered as the quality that is optimized in natural life.

How is a population with increasing fitness generated?

- **Let us consider a population of rabbits**. Some rabbits are faster than others, and we may say that these rabbits possess superior fitness, because they have a greater chance of avoiding foxes, surviving and then breeding.
- If two **parents** have superior fitness, there is a good chance that a **combination of their genes** will produce an **offspring** with even **higher fitness**.
- Over time the entire population of rabbits becomes faster to meet their environmental challenges in the face of foxes.

Simulation of Natural Evolution

- All methods of **evolutionary computation** ***simulate natural evolution*** by:
  - (A) creating a population of individuals,
  - (B) evaluating their fitness,
  - (C) generating a new population through genetic operations, and
  - (D) repeating this process a number of times.

- We will talk about **Genetic Algorithms (GAs)** as most of the other evolutionary algorithms can be viewed as variations of GAs.

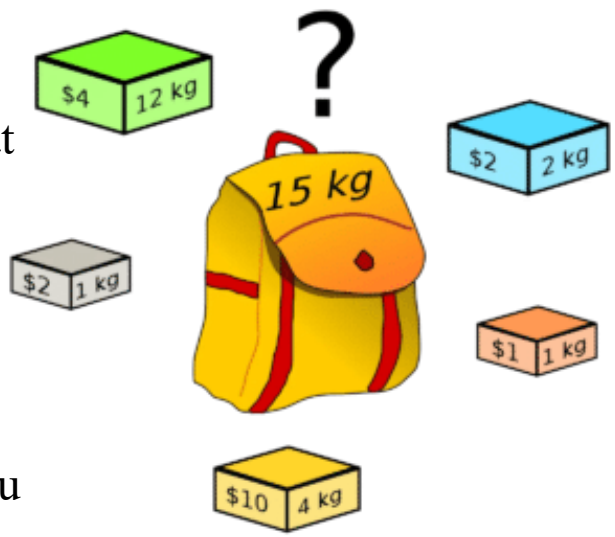
The Backpack Problem

Imagine you are going on a trip, and you have a **backpack** that can only hold a limited amount of weight. You also have a set of items you might want to pack.

- Each item has a **weight** (how heavy it is).
- Each item has a **value** (how useful or important it is to you).

Your challenge:

You want to choose a set of items to put in your backpack so that the **total weight does not go over the limit**, but the **total value is as high as possible**.



Think About:

1. How would you explain this problem in your own words?
2. If you were faced with this situation, what strategy might you try first?
   - Would you just pick the most valuable items?
   - The lightest ones?
   - Try all combinations?
   - Or something else?
3. What difficulties might you run into when trying to solve it?
4. Can you think of a **real-life example** (besides packing a backpack) where you have to make choices under limits like this?

Genetic Algorithms

- In the early 1970s, **John Holland** introduced the concept of **genetic algorithms.**

- His aim was to make computers do what nature does. **Holland** was concerned with algorithms that manipulate strings of binary digits.

- **GA can be represented** by a sequence of procedural steps for **moving from one population of artificial 'chromosomes' to a new population**.

- Each artificial "**chromosomes**" consists of a **number of "genes",** and each gene is represented by 0 or 1:

1	0	1	1	0	1	0	0	0	0	0	1	0	1	0	1

A 16-bit binary string of an artificial chromosome

- Nature has an ability to adapt and learn without being told what to do. In other words, **nature finds good chromosomes blindly**. GAs do the same.

- **Two mechanisms link a GA to the problem it is solving**: **encoding** and **evaluation**.
  - **Encoding** is carried out by representing chromosomes as strings of ones and zeros.
  - **An evaluation function** is used to measure the chromosome's performance, or fitness, for the problem to be solved

- The GA uses a measure of fitness of individual chromosomes to carry out reproduction. As **reproduction** takes place,
  - The **crossover** operator **exchanges parts of two single chromosomes**, and
  - The **mutation** operator **changes the gene value in some randomly chosen location of the chromosome**.

- As a result, **after a number of successive reproductions**, the less fit chromosomes become extinct, while those best able to survive gradually come to dominate the population. It is a simple approach, yet even crude reproduction mechanisms display highly complex behavior and are capable of solving some difficult problems.

What is GA

- A **genetic algorithm** (or **GA**) is a search technique used in computing to find true or approximate solutions to optimization and search problems.
- Genetic algorithms are categorized as global search heuristics.
- Genetic algorithms are a particular class of evolutionary algorithms that use techniques inspired by evolutionary biology such as inheritance, mutation, selection, and crossover (also called recombination).

What is GA

- Genetic algorithms are implemented as a computer simulation in which a population of abstract representations (called chromosomes or the genotype or the genome) of candidate solutions (called individuals, creatures, or phenotypes) to an optimization problem evolves toward better solutions.

- Traditionally, solutions are represented in binary as strings of 0s and 1s, but other encodings are also possible.

What is GA

- The evolution usually starts from a population of randomly generated individuals and happens in generations.

- In each generation, the fitness of every individual in the population is evaluated, multiple individuals are selected from the current population (based on their fitness), and modified (recombined and possibly mutated) to form a new population.

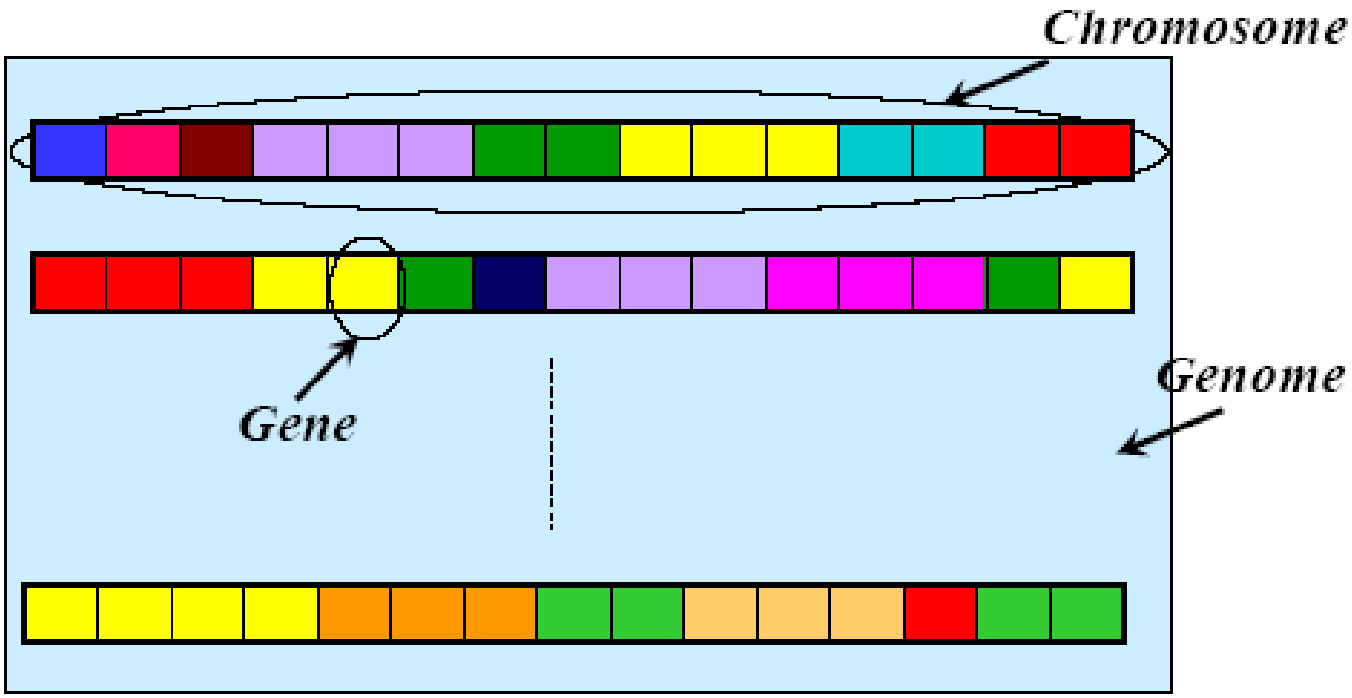
What is GA

- The new population is then used in the next iteration of the algorithm.
- Commonly, the algorithm terminates when either a maximum number of generations has been produced, or a satisfactory fitness level has been reached for the population.
- If the algorithm has terminated due to a maximum number of generations, a satisfactory solution may or may not have been reached.

Key terms

- **Individual** - Any possible solution
- **Population** - Group of all **individuals**
- **Search Space** - All possible solutions to the problem
- **Chromosome** - Blueprint for an **individual**
- **Trait** - Possible aspect **(features)** of an **individual**
- **Allele** - Possible settings of trait (black, blond, etc.)
- **Locus** - The position of a **gene** on the **chromosome**
- **Genome** - Collection of all **chromosomes** for an **individual**

Chromosome, Genes and Genomes

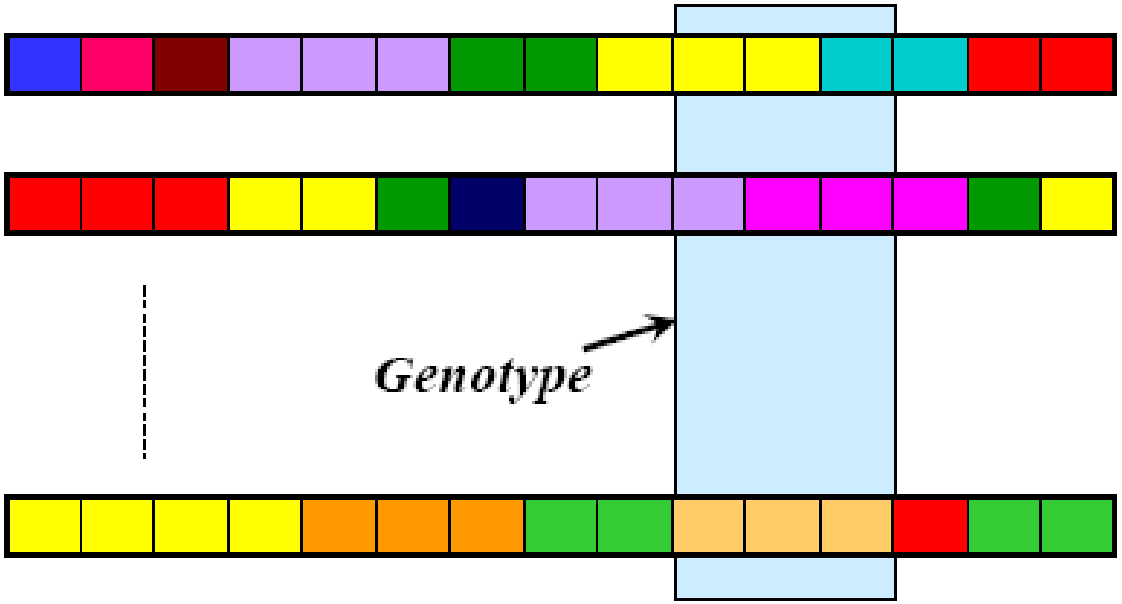


Genotype and Phenotype

- **Genotype:**
  – Particular set of genes in a genome

- **Phenotype:**
  – Physical characteristic of the genotype (smart, beautiful, healthy, etc.)

Genotype and Phenotype



GA Requirements

- A typical genetic algorithm requires two things to be defined:
  - a genetic representation of the solution domain, and
  - a fitness function to evaluate the solution domain.

- A standard representation of the solution is as an array of bits. Arrays of other types and structures can be used in the same way.

- The main property that makes these genetic representations convenient is that their parts are easily aligned due to their fixed size, that facilitates simple crossover operation.

- Variable length representations may also be used, but crossover implementation is more complex in this case.

Representation

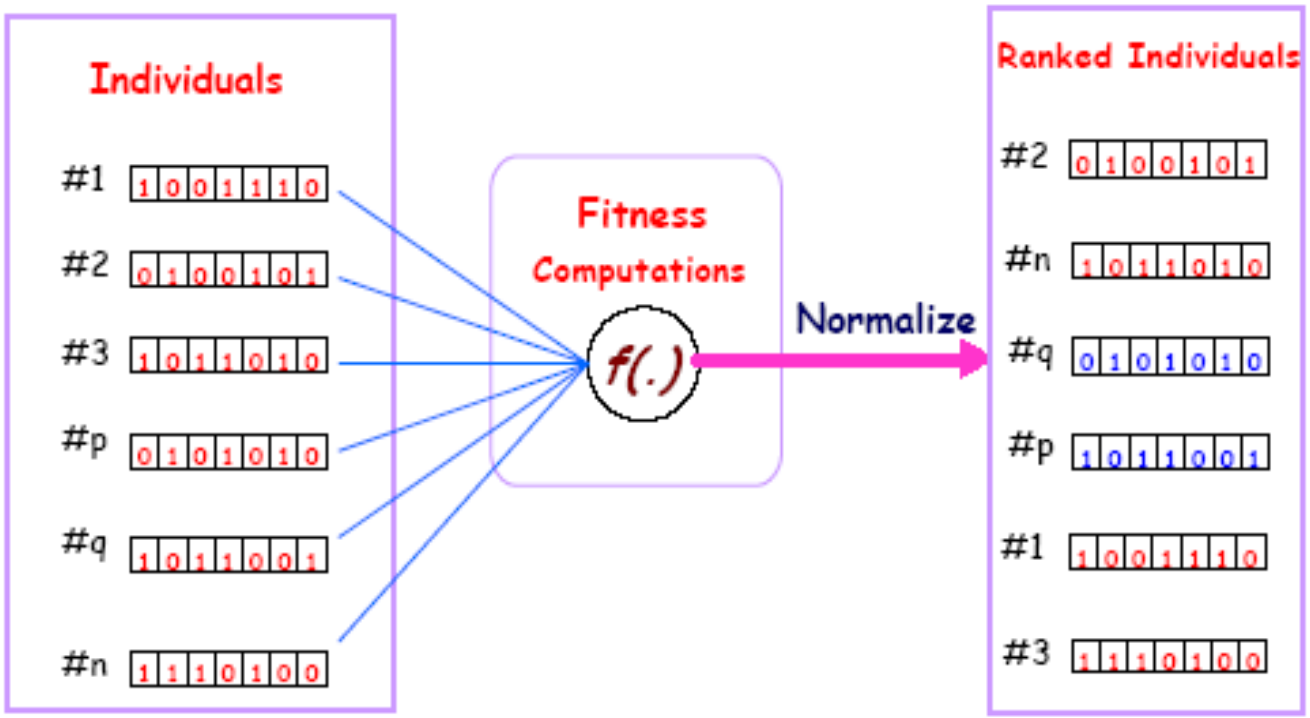
Chromosomes could be:

- Bit strings (0101 ... 1100)
- Real numbers (43.2 -33.1 ... 0.0 89.2)
- Permutations of element (E11 E3 E7 ... E1 E15)
- Lists of rules (R1 R2 R3 ... R22 R23)
- Program elements (genetic programming)
- ... any data structure ...

GA Requirements

- The fitness function is defined over the genetic representation and measures the **quality** of the represented solution.
- The fitness function is always problem dependent.
- For instance, in the **knapsack problem** we want to maximize the total value of objects that we can put in a knapsack of some fixed capacity.
- A representation of a solution might be an array of bits, where each bit represents a different object, and the value of the bit (0 or 1) represents whether or not the object is in the knapsack.
- Not every such representation is valid, as the size of objects may exceed the capacity of the knapsack.
- The **fitness** of the solution is the sum of values of all objects in the knapsack if the representation is valid, or 0 otherwise. In some problems, it is hard or even impossible to define the fitness expression; in these cases, interactive genetic algorithms are used.

A fitness function



Basic Genetic Algorithms

Step 1: Represent the problem variable domain as a chromosome of a fixed length, choose the size of a chromosome population N, the crossover probability p_c and the mutation probability p_m.

Step 2: Define a **fitness function** to measure the performance, or fitness, of an individual chromosome in the problem domain. The fitness function establishes the basis for selecting chromosomes that will be mated during reproduction.

Step 3: Randomly generate an initial population of chromosomes of size N:

$\boldsymbol{x_1, x_2, \ldots, x_N}$

Step 4: Calculate the fitness of each individual chromosome:

$\boldsymbol{f(x_1), f(x_2), \ldots, f(x_N)}$

Step 5: Select a pair of chromosomes for mating from the current population. Parent chromosomes are selected with a probability related to their fitness.

Step 6: Create a pair of offspring chromosomes by applying the genetic operators – **crossover** and **mutation**.

Step 7: Place the created offspring chromosomes in the new population.

Step 8: Repeat *Step 5* until the size of the new chromosome population becomes equal to the size of the initial population, *N*.

Step 9: Replace the initial (parent) chromosome population with the new (offspring) population.

Step 10: Go to *Step 4*, and repeat the process until the termination criterion is satisfied.

Notes

- **GA represents an iterative process**. Each iteration is called a **generation**. A typical number of generations for a simple GA can range from 50 to over 500. The entire set of generations is called a run.

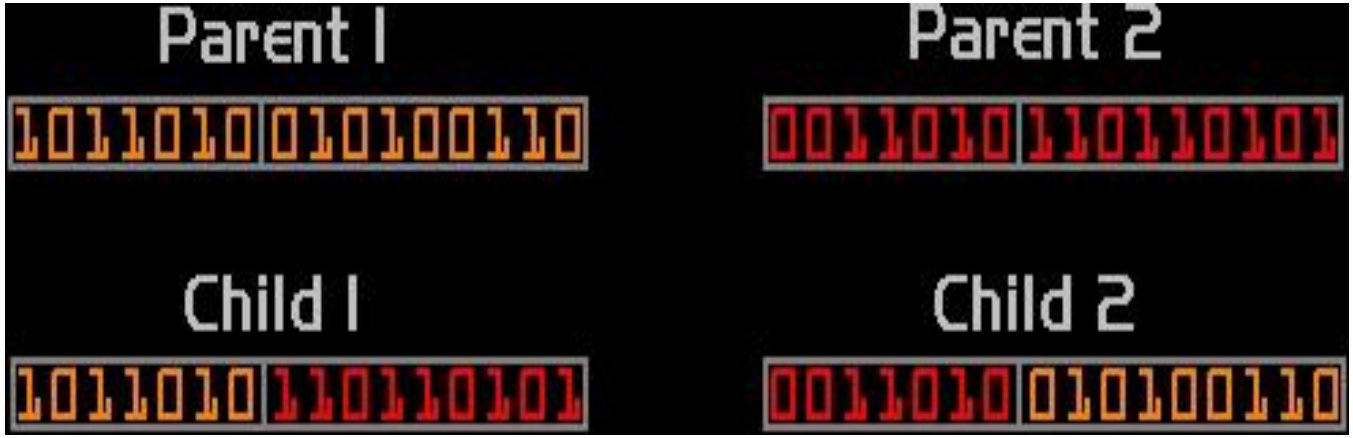
- Because GAs use a stochastic search method (randomly determined), the fitness of a population may remain stable for a number of generations before a superior chromosome appears.

- A common practice is to terminate a GA after a specified number of generations and then examine the best chromosomes in the population. **If no satisfactory solution is found, the GA is restarted.**

Crossover

- the most common type is single point crossover. In single point crossover, you choose a locus at which you swap the remaining alleles from on parent to the other. This is complex and is best understood visually.
- As you can see, the children take one section of the chromosome from each parent.
- The point at which the chromosome is broken depends on the randomly selected crossover point.
- This particular method is called single point crossover because only one crossover point exists. Sometimes only child 1 or child 2 is created, but oftentimes both offspring are created and put into the new population.
- Crossover does not always occur, however. Sometimes, based on a set probability, no crossover occurs and the parents are copied directly to the new population. The probability of crossover occurring is usually 60% to 70%.

Crossover


Parent 1
1011010 010100110
Parent 2
0011010 110110101
Child 1
1011010 110110101
Child 2
0011010 010100110

Mutation

- After selection and crossover, you now have a new population full of individuals.
- Some are directly copied, and others are produced by crossover.
- In order to ensure that the individuals are not all exactly the same, you allow for a small chance of mutation.
- You loop through all the alleles of all the individuals, and if that allele is selected for mutation, you can either change it by a small amount or replace it with a new value. The probability of mutation is usually between 1 and 2 tenths of a percent.
- Mutation is fairly simple. You just change the selected alleles based on what you feel is necessary and move on. Mutation is, however, vital to ensuring genetic diversity within the population.

Mutation

GA Pseudo-code

Choose initial population

Evaluate the fitness of each individual in the population

Repeat

Select best-ranking individuals to reproduce

Breed new generation through crossover and mutation (genetic operations) and give birth to offspring

Evaluate the individual fitnesses of the offspring

Replace worst ranked part of population with offspring

Until <terminating condition>

Genetic Algorithms: Example

A simple example will help us to understand how a GA works. Let us find the **maximum value** of the function $(15x - x^2)$ where parameter x varies between 1 and 15. For simplicity, we may assume that x takes only integer values.

Thus, **chromosomes** can be built with only **four genes:**

Integer	*Binary code*	*Integer*	*Binary code*	*Integer*	*Binary code*
1	0 0 0 1	6	0 1 1 0	11	1 0 1 1
2	0 0 1 0	7	0 1 1 1	12	1 1 0 0
3	0 0 1 1	8	1 0 0 0	13	1 1 0 1
4	0 1 0 0	9	1 0 0 1	14	1 1 1 0
5	0 1 0 1	10	1 0 1 0	15	1 1 1 1

The Fitness Function and Chromosome Locations

Suppose that:
the size of the chromosome population N is 6,
the crossover probability p_c equals 0.7, and
the mutation probability p_m equals 0.001.

(The values chosen for p_c and p_m are fairly typical in GAs.)

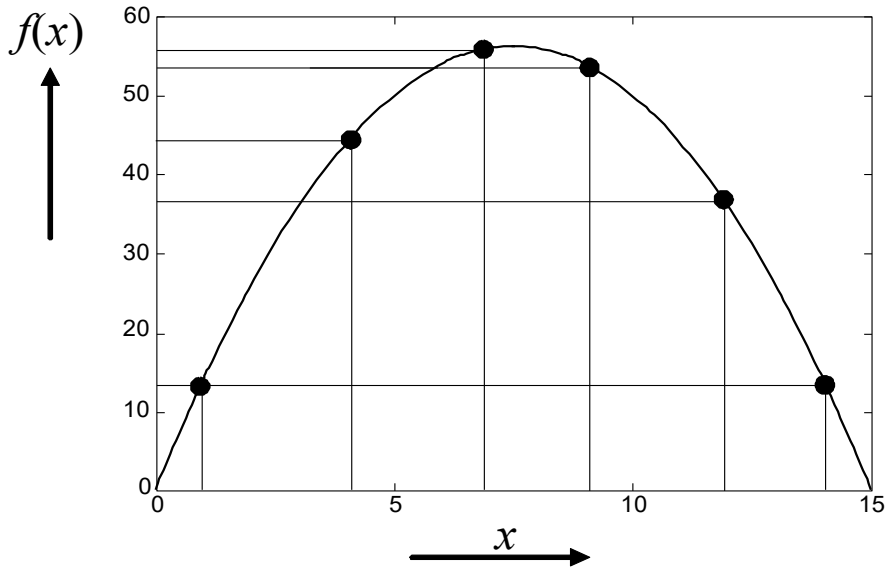
The fitness function in our example is defined by:

$$F(x) = 15\,x - x^2$$

The GA creates an initial population of chromosomes by filling six 4-bit strings with randomly generated ones and zeros. The initial population might look like:

Chromosome label	*Chromosome string*	*Decoded integer*
X1	1 1 0 0	12
X2	0 1 0 0	4
X3	0 0 0 1	1
X4	1 1 1 0	14
X5	0 1 1 1	7
X6	1 0 0 1	9

The next step is to **calculate the fitness of each individual chromosome**. The chromosomes' initial locations on the fitness function are illustrated as:



Chromosome initial locations.

A real practical problem would typically have a population of thousands of chromosomes.

Chromosome label	*Chromosome string*	*Decoded integer*	*Chromosome fitness*	*Fitness ratio, %*
X1	1 1 0 0	12	36	16.5
X2	0 1 0 0	4	44	20.2
X3	0 0 0 1	1	14	6.4
X4	1 1 1 0	14	14	6.4
X5	0 1 1 1	7	56	25.7
X6	1 0 0 1	9	54	24.8

The average fitness of the initial population is 36. In order to improve it, the initial population is modified by using selection, crossover and mutation, the genetic operators.

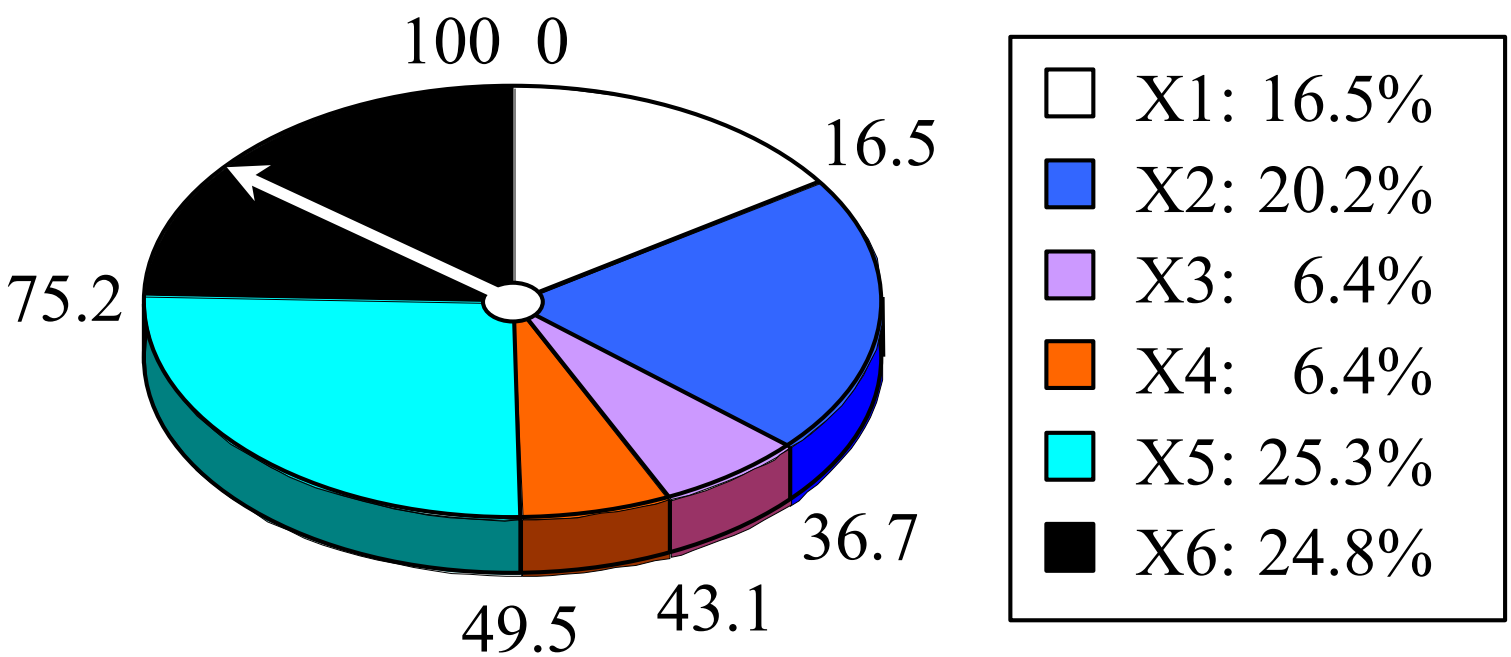
- **In natural selection**, only the fittest species can survive, breed, and thereby pass their genes on to the next generation.
  - **GAs** use a similar approach, but unlike nature, the **size** of the chromosome population **remains unchanged** from one generation to the next.

How can we maintain the size of the population constant, and at the same time improve its average fitness?

- The last column in the table shows the ratio of the individual chromosome's fitness to the population's total fitness.
  - **This ratio determines the chromosome's chance of being selected for mating**.
  - Thus, the chromosomes **X5** and **X6** stand a **fair chance**, while the chromosomes **X3** and **X4** have **a very low probability of being selected**.
  - As a result, The chromosome's average **fitness improves from one generation to the next**.

Roulette Wheel Selection

The most commonly used chromosome selection techniques is the **roulette wheel selection**.



- As you can see, each chromosome is given a slice of a circular roulette wheel. The area of the slice within the wheel is equal to the chromosome fitness ratio.
  - For instance, the chromosomes X5 and X6 (the most fit chromosomes) occupy the largest areas, whereas the chromosomes X3 and X4 (the least fit) have much smaller segments in the roulette wheel.

- **To select a chromosome for mating**, a **random number** is generated in the **interval [0,100]**, and the chromosome whose segment spans the random number is selected.

- It is like spinning a roulette wheel where each chromosome has a segment on the wheel proportional to its fitness.

- The roulette wheel is spun, and **when the arrow comes to rest on one of the segments**, the corresponding **chromosome** is selected.

- In our example, we have an **initial population of 6 chromosomes**. Thus, to establish the same population in the next generation, the **roulette wheel would be spun six times**.
  - The **first two spins** might select chromosomes **X6 and X2** to become parents, the **second pair** of spins might choose chromosomes **X1 and X5**, and the **last two** spins might select chromosomes **X2 and X5**.

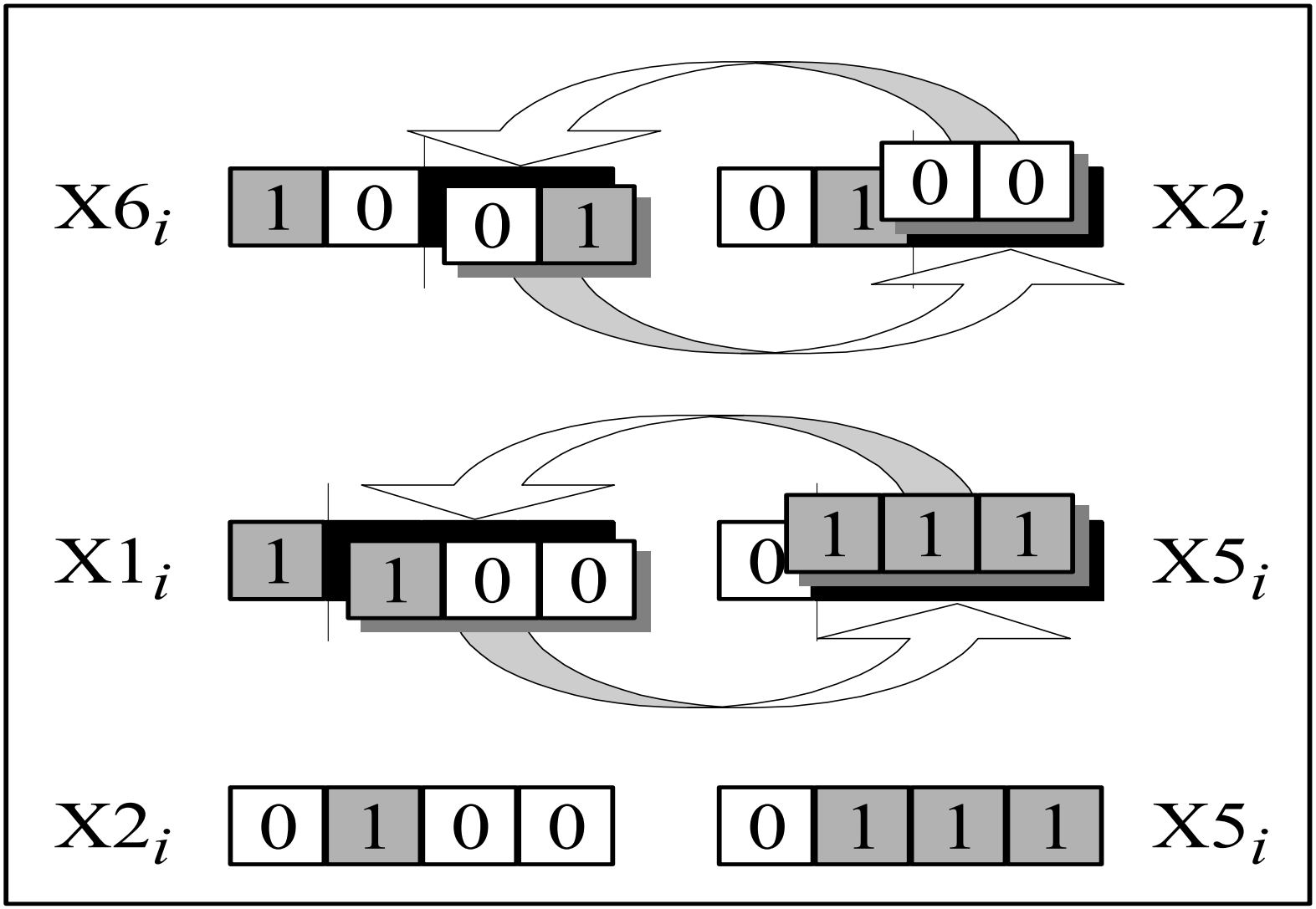
- **Once a pair of parent chromosomes is selected**, the **crossover** operator is applied.

Crossover Operator

- First, **the crossover operator randomly chooses a crossover point where two parent chromosomes "break", and then exchanges the chromosome parts after that point**. As a result, two new offspring are created.
  - For example, the chromosomes X6 and X2 could be crossed over after the second gene in each to produce the two offspring.

- If a pair of chromosomes does not cross over, then the chromosome cloning takes place, and the offspring are created as exact copies of each parent.
  - For example, the parent chromosomes X2 and X5 may not cross over. Instead, they create the offspring that are their exact copies.

Crossover



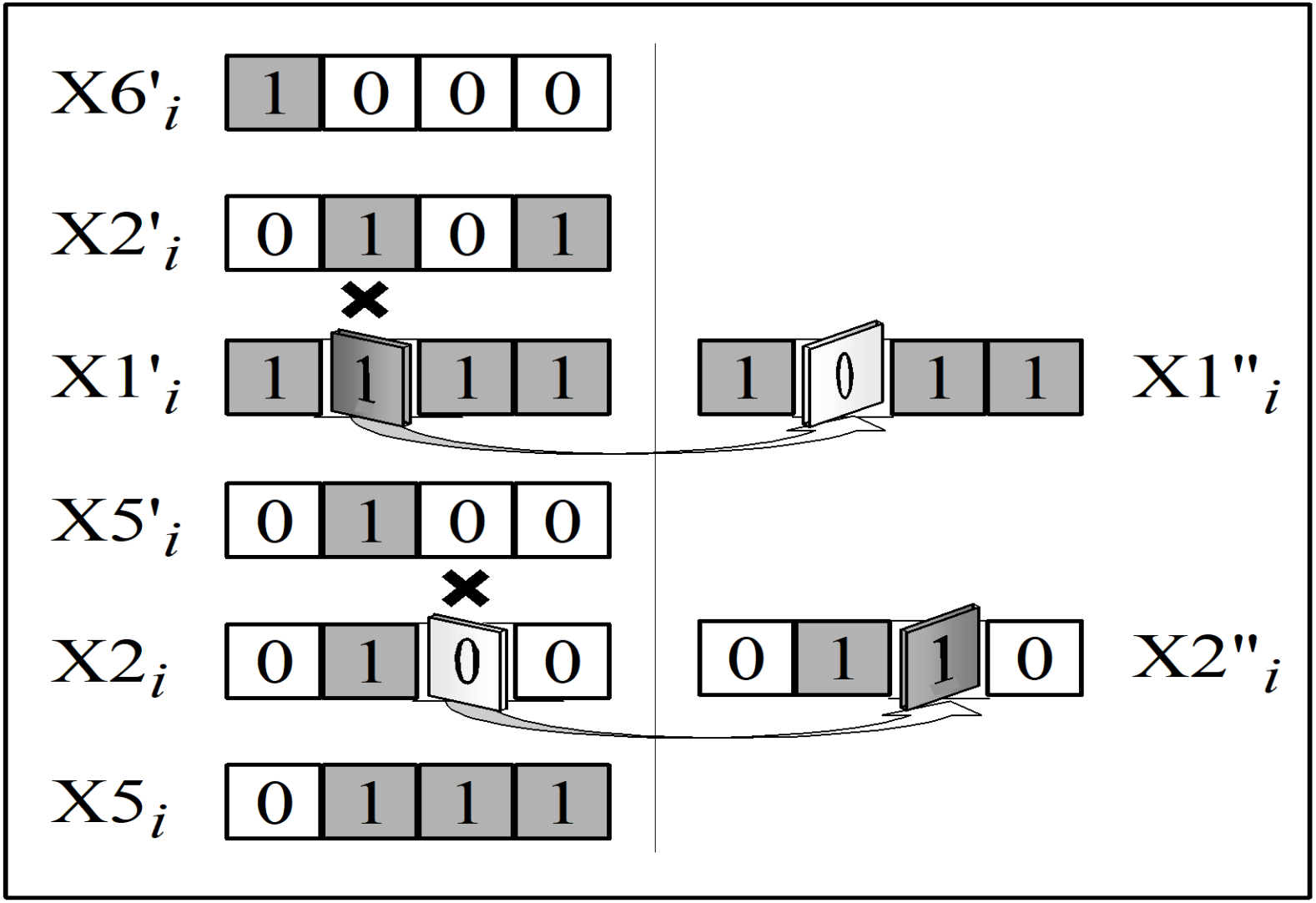
- A value of 0.7 for the crossover probability generally produces good results.

- **After selection and crossover**, the average fitness of the chromosome population has improved and gone from 36 to 42.

Mutation Operator

- **Mutation**, which is rare in nature, represents a change in the gene. It may lead to a significant improvement in fitness, but more often has rather harmful results.
- Mutation is a background operator. Its role is to provide a guarantee that the search algorithm is not trapped on a local optimum.
- The mutation operator **flips a randomly selected gene in a chromosome**.
- **The mutation probability is quite small in nature**, and is kept low for GAs, typically in the range between 0.001 and 0.01.

Mutation


X6'i
1 0 0 0
X2'i
0 1 0 1
X1'i
1 1 1 1
1 0 1 1
X1"i
X5'i
0 1 0 0
X2i
0 1 0 0
0 1 1 0
X2"i
X5i
0 1 1 1

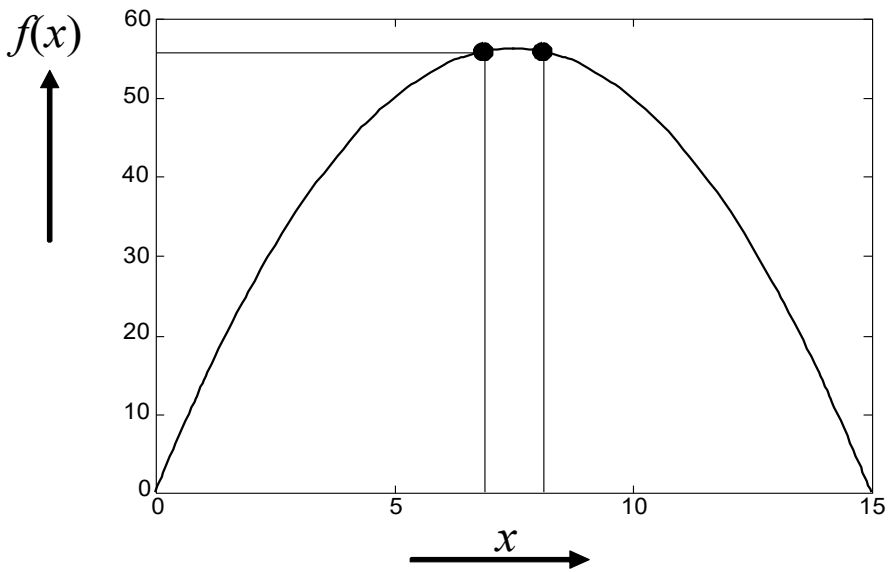
Generation $(i + 1)$

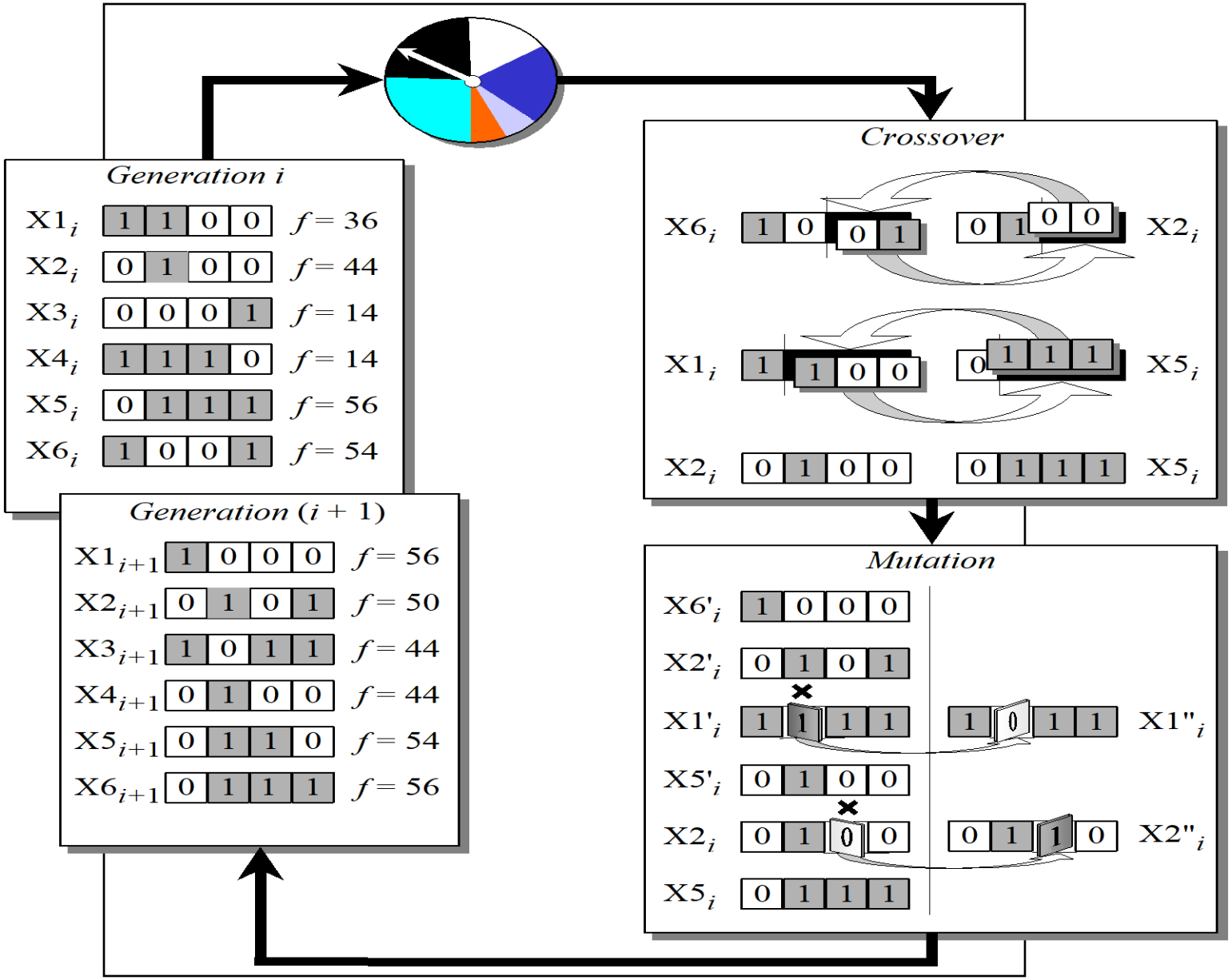
$X1_{i+1}$	1	0	0	0	$f = 56$
$X2_{i+1}$	0	1	0	1	$f = 50$
$X3_{i+1}$	1	0	1	1	$f = 44$
$X4_{i+1}$	0	1	0	0	$f = 44$
$X5_{i+1}$	0	1	1	0	$f = 54$
$X6_{i+1}$	0	1	1	1	$f = 56$

- Genetic algorithms assure the continuous improvement of the average fitness of the population, and after a number of generations (typically several hundred) the population evolves to a **near-optimal** solution.

- In our example, the final population would consist of only chromosomes

$$\boxed{0\,1\,1\,1} \text{ and } \boxed{1\,0\,0\,0}.$$



Chromosome final locations

The Genetic Algorithm Cycle